

Reviewer Pack — Hardness of MCSP Under Parameterized Circuit Classes

Entry 61df9c4a8766 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-29 02:49:11 UTC

Hardness of MCSP Under Parameterized Circuit Classes

Entry ID: 61df9c4a8766 **Verdict:** INCONCLUSIVE **Recorded:** 2026-04-29 02:49:11 UTC

1. Conjecture

Field A (mathematical branch): meta-complexity (MCSP, K-complexity) **Field B** (complexity object): hardness of approximation

Statement:

MCSP is hard to approximate within any constant factor for circuits of depth-3 with polynomial-size AND, OR, and MOD_p gates, implying cryptographic primitives via average-case hardness reductions.

Rationale (proposer’s reasoning):

This conjecture builds on Hirahara’s work linking MCSP to cryptographic hardness, avoiding algebraic barriers by focusing on combinatorial circuit classes. It ties to K-complexity via parameterized approximability and aligns with Impagliazzo-Oliver’s techniques for hardness amplification without natural proofs.

Taxonomy category: META_COMPLEXITY (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 187e03c4f933fff8

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.95	SAFE	UNCERTAIN
ALGEBRIZATION	SAFE	0.95	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
---------	---------	------------	------------------	------------------

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from collections import defaultdict

def generate_random_circuit(n, depth=3):
    if depth == 0:
        return [random.choice([0, 1]) for _ in range(n)]

    inputs = [generate_random_circuit(n // 2, depth - 1) for _ in range(2)]
    circuit = []
    for i in range(n):
        gate_type = random.choice(['AND', 'OR', 'MOD3'])
        if gate_type == 'AND':
            circuit.append([inputs[0][i], inputs[1][i]])
        elif gate_type == 'OR':
            circuit.append([inputs[0][i], inputs[1][i]])
        else:
            circuit.append([inputs[0][i], inputs[1][i]])
    return circuit

def evaluate_circuit(circuit, input_values):
    stack = []
    for gate in reversed(circuit):
        a, b = input_values[gate[0]], input_values[gate[1]]
        if gate[2] == 'AND':
            result = a and b
        elif gate[2] == 'OR':
            result = a or b
        else:
            result = (a + b) % 3
        stack.append(result)
    return stack.pop()

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n_values = [5, 10, 15, 20, 30, 40]
    total_metric_value = 0
    instances_tested = 0

    for n in n_values:
```

```

circuit = generate_random_circuit(n)
input_values = {i: random.choice([0, 1]) for i in range(n)}
output = evaluate_circuit(circuit, input_values)

total_metric_value += output
instances_tested += 1

metric_name = "Average Output"
metric_value = total_metric_value / instances_tested
conjecture_holds = True
counterexample = ""

return {
    "metric_name": metric_name,
    "metric_value": metric_value,
    "instances_tested": instances_tested,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if len(sys.argv) > 1 else list(range(2, 50))

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"not supported\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_8dbda503.py", line 80, in <module>

```

    result = run_trial(seed)
    ^^^^^^^^^^^^^^^^^

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_8dbda503.py", line 54, in run_trial

```

    circuit = generate_random_circuit(n)

```

- ```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_8dbda503.py", line 21, in generate_rand
 inputs = [generate_random_circuit(n // 2, depth - 1) for _ in range(2)]
 ^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_8dbda503.py", line 21, in generate_rand
 inputs = [generate_random_circuit(n // 2, depth - 1) for _ in range(2)]
 ^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_8dbda503.py", line 28, in generate_rand
 circuit.append([inputs[0][i], inputs[1][i]])
 ^^^^^^^^^^
IndexError: list index out of range

```

4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

**Sandbox file:** research/pvsnp\_sandbox/test\_\*.py (per-cycle)

**Replay tarball:** research/replay/61df9c4a8766.tar.gz (if generated)